Artificial Intelligence - Navigating a dangerous environment, using Djikstra and Q-learning

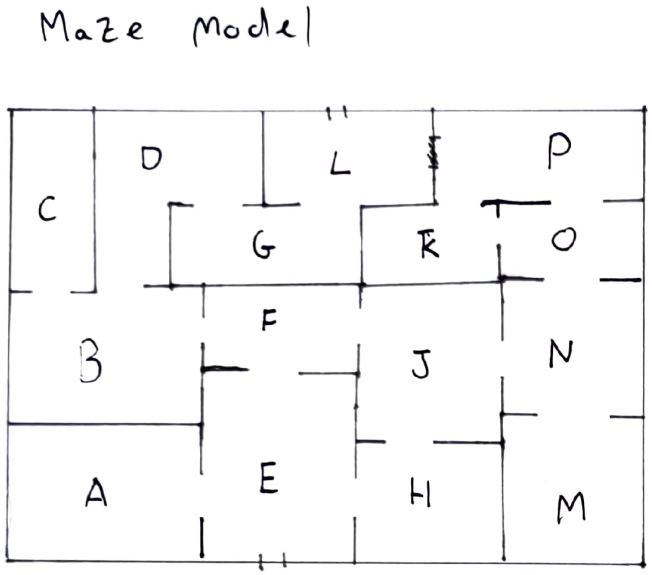
Abstract

This paper will explore and evaluate how the use of Dijkstra's algorithm and Q-learning can be used in navigating a maze-like environment. There are many different algorithms that can find the optimum route through a maze, such as breadth first search and depth first search. In this particular scenario, there is no ability to pre-map the room, the robot/agent must physically move around the environment to achieve the goal (exit). In this report we have evaluated that Dijkstra is more time and space efficient when finding the optimum route and can map terrain by accounting time taken to travel to node in calculation. Both have a similar chance of "stumbling" across the exit if we are searching for the quickest time to receive a viable exit. But in contrast, Q-learning is more advantageous if environmental conditions are evaluated. By predetermining that moving towards smoke is a negative reward and "cleaner air" a more positive reward, Q-learning can increase its efficiency. Furthermore studies by M. Pieters and M. A. Wiering((M. Pieters and M. A. Wiering, 2016), show that Q-learning can be adapted for a dynamic environment, which is advantageous for a building with falling debris.

Define the problem

Path finding is a problem that we all encounter, whether it's travelling from A to B or part of an overarching idea applied in the programming of our technology. There are various different algorithmic approaches and processes that can be employed to achieve pathfinding. The example that will be discussed in this paper is the evaluation of Dijkstra's algorithm and reinforced learning, specifically Q-learning (Watkins, 1989). The scenario that these two processes will be compared in is a scalable changeable environment.

Specifically, a room or a series of rooms with obstacles that might change over time. Here a robot will be utilised to navigate a building with autonomy, debris may fall from the ceiling changing the maze-like environment. AI is providing an ability to protect ourselves, to achieve progress without human endangerment. An example of this is outlined by Bird in their paper on the use of autonomous robots to monitor nuclear facilities (Bird et a, 2019). The aim of this evaluation is to see how appropriate both algorithmic processes would be in a real life situation where say there is a burning building and there is no way to pre-map any of the rooms or scan the rooms. To start with let us also assume that the rooms are not changing, and our robot can also easily physically navigate the environment. There are two versions of this scenario, firstly there is no time pressure where the robot has time to run it's process and to return to the optimal path, and secondly, as soon as the robot has completed one episode (reached the goal) the route is returned. From this point, a human can now enter the building taking the respective route to reach the goal.



How are we going to compare these? First we will analyse the processes of Q-learning and Djkstra in a small, medium, and large environment comparing efficiency of returning a viable route and then returning an optimal route. Considering also how we can manipulate our Q learning code to “avoid” implications of danger. For example; more negative reward for smoke, more positive the more light sensed. Finally, let’s consider the possibility of a changeable environment. How would we be able to alter our code to deal with the new appearances of obstacles?

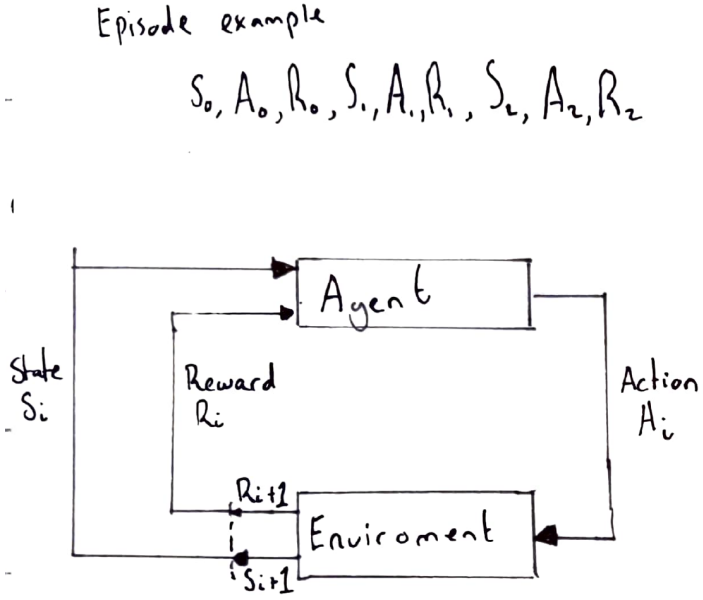
Before we can make any evaluation on how these processes interact, we must fundamentally understand how they work. Followed by how we have approached coding a scalable environment.

Q-learning

Before approaching our environment, let’s cover in detail how Q-learning works. Reinforcement learning is a branch of machine learning, with reinforcement learning you will find the model of Q-learning. Q-learning is a procedure where an agent interacts with an environment to achieve a goal, this could be an action goal or physical goal. When the agent interacts with the environment via trial and error, if the agent makes a “good” move it is rewarded positively and if the agent makes a “bad” move a negative reward is implemented. Note, what is a “good” or “bad” move must be defined; the agent cannot decide what is intrinsically good or bad. The reward policy is scalar with the goal having the highest positive reward. The ultimate goal of the agent is to maximise its cumulative expected reward .

The agent will pass through episodes, and each episode loops through state-action-reward. State-action-reward is the process of returning the agent's state (position in the environment), passing an action phase then passing a reward phase where the agent will receive a positive or

negative reward. State-action-reward will loop until the agent reaches the designated goal and the episode is ended.



In order for the agent to 'learn' about the environment, the state and action of the agent is mapped onto a matrix called the Q-table. This Q-table is then updated at the end of an episode to create a map of possible moves for the agent. There are two ways in which the agent can interact with the environment. Firsty via exploitation; the agent, using the q-table as a reference, can view all possible actions for a given state. From this the agent chooses the action which returns the highest reward. The second method, exploring, is for the agent to act stochastically. In order for the agent to explore more of the environment, and calculate/discover potential shorter routes, the agent ignores the informed q-table and selects an action at random. Epsilon is the deterministic value in which the proportion of exploitation to exploration is decided.

Here we can use the Bellman equation to help the learning process. Without it, if the agent were to change starting position it would not be able to find a path towards the goal as all states have been set to an equal positive reward. The Bellman equation equates the reward that the agent receives for the current state s0 plus the maximum reward for the next state, as the maximum future reward.

$$V(s) = max(a) *(R(s,a) + gamma * V(s_1))$$

V(s) = The expected return Value at the current state
max(a)= maximum value for any action
R(s,a) = THe expected reward for the action a at state s
gamma * V(s1)=the discount factor m multiplice the return value of the next state

From this principle we can use this logic to create an equation that will update the Q-table:

Q[state, action]

= Q[state, action] + lr * (reward + gamma * np.max(Q[new_state, :]) - Q[state, action])

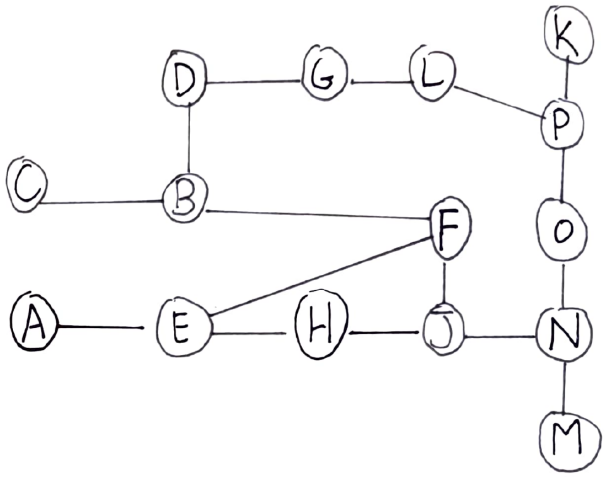
lr(***α***) = Learning rate(alpha)
gamma(***γ***) = discount factor
np.max(Q[new_state, :])=estimate of optimal future value
reward + gamma * np.max(Q[new_state, :]) = the learned value

Dijkstra

Dijkstra's algorithm is a path finding algorithm created by Dr. Edsger W. Dijkstra and published in 1959. This relatively simple algorithm is used widely throughout modern navigation software.

"One of the reasons that it is so nice was that I designed it without pencil and paper. Without pencil and paper you are almost forced to avoid all avoidable complexities."
(page 43, Frana and Misa, 2010)

To understand Dijkstra's algorithm we must use graphs as our data structure. Each actionspace/object in our environment is marked by a node and these nodes are connected via edges. Dijkstra's algorithm also takes into consideration the directionality of the edges and their weight. Dijkstra also uses a greedy policy, where it continues to explore even after finding the goal to find the optimum route. The algorithm keeps track of the currently known shortest distance (with the addition of the weight of the edge), storing it in a list, which is then updated if a shorter path is found. Two sets are created for visited nodes and unvisited nodes, and when a node is visited it is removed from the unvisited and placed in the visited. For each step between the nodes a cost is calculated by the addition of the edge's weight. Once the unvisited list is empty, the algorithm is finished. (Below graph example)



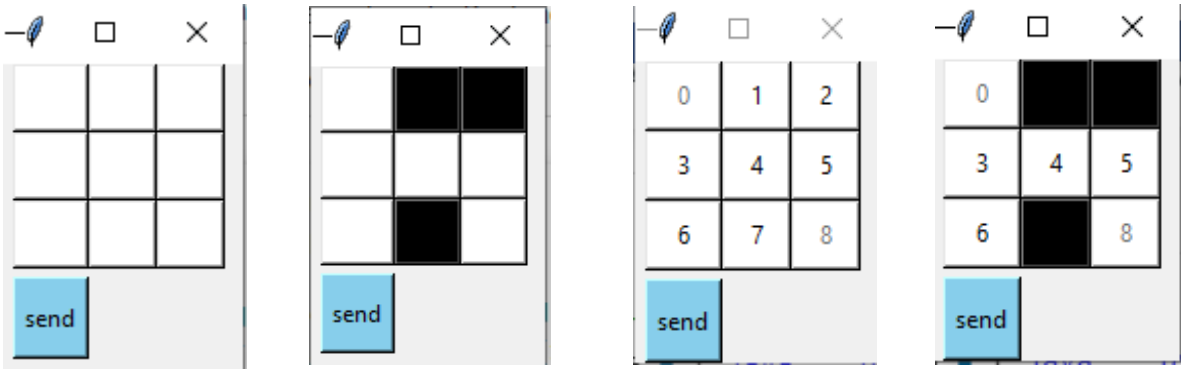
Environment

For this analysis, a simple user-friendly interface has been created to test various factors of how Djkstras and Q-learning algorithms work.

Ideally when approaching code for creating an agent in an environment, programming in an object oriented manner would be optimal. For my own coding simplicity, instead I have mapped a grid layout using a tkinter frame. The user can select cells to mark obstacles in the maze, and from this we can create a graph. Marking each cell as a node on the grid, for now we will negate the idea of difficult terrain or longer distances between the nodes

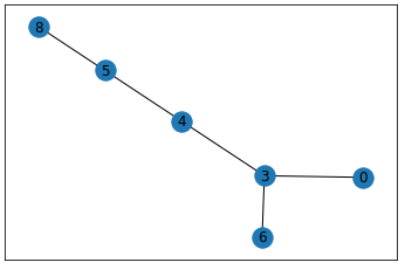
The initial thought process was to map out every white box by coordinate and then map these coordinates in a more useful format (e.g. nodes) like the example seen in the graph above. Instead a simpler solution is to have each button numbered physically with text. Then using cget[“text”] == [number_cell] I can call each button respectively .

Applying this to the code that I’ve written, let us start with a really simple 3 by 3 grid as seen in figure.2. Figure.3 shows a simple “maze” created by blocked out cells. Let us label each cell with some text to identify nodes at this non-coding point.




```
points_list = [(0,3),(3,6),(3,4),(4,5),(5,8)]
goal = 8

import networkx as nx
G=nx.Graph()
G.add_edges_from(points_list)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G,pos)
nx.draw_networkx_edges(G,pos)
nx.draw_networkx_labels(G,pos)
plt.show()
```



Without running any code, the solution to the naked eye is obvious; [0,3,4,5,8]. Once this grid is scaled up the shortest route of escape will no longer be obvious.

Let is now see what this would look like as a node map for visualisation processes to understand the Q learning principle. Without running any code let's note what our nodes will be. We can move between 0 to 3 (represented as (0,3)) from 3 we can go to 4 and 6, also represented as (3,4) and (3,6), ect. Producing a node list of [(0,3),(3,6),(3,4),(4,5),(5,8)].

In order to build a matrix for our algorithms we need to make sure the numbers of our nodes are all within a range

and not missing any values. This is because the matrix is run sequentially. Using numbers printed on every cell in the tkinter interface means in this example we're missing a node 1,2, and 7. This will not work when building a matrix.

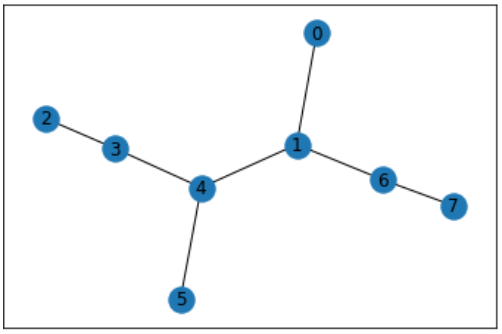
Below is a working example, to show why we need to label our nodes in this way.

Note the use of rewards in the matrix.
A viable move == 0
An non viable move (e.g an non adjacent node) == -1
The goal point == 100

Also, the cell that the agent is currently on is also set to -1 to discourage revisiting.

This is an example of how the matrix works [(0,1), (1,4), (4,3),(4,5),(3,2),(1,6),(6,7)]

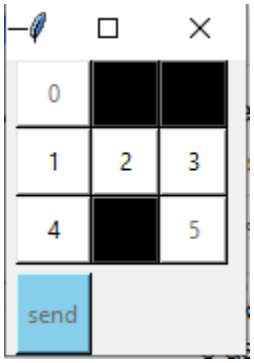
```
matrix([[ -1.,   0.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [  0.,  -1.,  -1.,  -1.,   0.,  -1.,   0.,  -1.],
        [ -1.,  -1.,  -1.,   0.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,   0.,  -1.,   0.,  -1.,  -1.,  -1.],
        [ -1.,   0.,  -1.,   0.,  -1.,   0.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,   0.,  -1.,  -1.,  -1.],
        [ -1.,   0.,  -1.,  -1.,  -1.,  -1.,  -1., 100.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,   0., 100.]])
```



Here the goal is set at node 7 starting at node 0. For each row in the matrix is an individual node's moves, so for node 0 there is only one viable path (move to node 1). This is reflected in the reward scheme moving to node 1 rewards 0 compared to -1 for any other move. For example if we look at node 4, which is row 5 in the matrix, there are no negative rewards for moving to 1, 3 or 5 as these are viable routes. If we were missing node numbers a matrix would not be able to be built using this algorithm as the for loop runs sequentially. This is why we need to label our nodes from the tkinter interface within a range of numbers so that our matrix can be calculated sequentially.

Another method to create the matrix suitable for the Q-learning algorithm is to set non-viable routes (e.g. wall/ borders) by setting them a large negative reward (e.g. -50).

This method that I have implemented excludes any non-viable nodes from the numbered cells. A benefit to this over the other method is the reduction of nodes running through the algorithm. Instead of a 9 by 9 matrix we are only (in this example) running a 6x6 matrix, slightly reducing the time complexity. Below shows the code for achieving this and the result on the user interface.



```
#lets at a unique number to every button
for button in range(len(button_ids)):
    button_ids[button]["text"]=button
```

Without writing any code we can see from the display we have a points list: [(0,1),(1,4),(1,2),(2,3),(3,5)] and when run through the Q-learning algorithm this successfully returns [0, 1, 2, 3, 5].

```
def finalise():
    numbers=[i for i in range(10)]
    for button in button_ids:
        button_colour = button.cget('bg')
        if button_colour == "white":
            white_spaces.append(button)
            button["text"] = str(numbers[0])
            numbers.pop(0)
    finalise_button["state"]="disabled"
```

As it stands our program currently returns each empty cell in a list of tuples where the tuples are coordinates (where in this example 0 is (0,0) and 5 is (2,). The challenge now is to convert this list into a viable format that can be processed into a matrix.

There is most probably a much more time and space efficient way to achieve this goal, perhaps skipping the production of coordinates in a list and being able to map the tkinter grid straight as a set of node paths. For my own coding simplicity, I have zipped the coordinates to their respective cell number in a dictionary {0: (0, 0), 1: (1, 0), 2: (1, 1), 3: (1, 2), 4: (2, 0), 5: (2, 2)} and used the following code to find the node paths by using an agent like model. For each coordinate in the list the agent looks up, down, right, and left to see if there is a viable space to move to. Each key for each coordinate is recorded in the points list.

For example starting at node 0:
look right search (i, i+1), there is no (0,1) so ignore
look left (i, i-1), outside of the border, no such coordinate, ignored
look up(i+1, i) outside of the border, no such coordinate, ignored
look down (i-1, i) (1,0) matches key 1 so add to node list as 0 to 1 (0,1)

And example of the code for this:

```
for key, value in dictonary.items():
    #finds the coords of current index
    x = value[0]
    y =value[1]
    firstkey = key
    #looks right from current  index see if there is a matching coord
    if [key for key, value in dictonary.items() if value == (x +1,y)]:
        right = [key for key, value in dictonary.items() if value == (x +1,y)]
        print(firstkey, right[0])
```

This produces [(0, 1), (1, 4), (1, 0), (1, 2), (2, 3), (2, 1), (3, 5), (3, 2), (4, 1), (5, 3)]
Let's filter any repeated connecting nodes out of this list for efficiency.

```
filtered_nodes = []
for node in node_points:
    if node not in filtered_nodes and node[::-1] not in filtered_nodes:
        filtered_nodes.append(node)
```

Producing [(0, 1), (1, 4), (1, 2), (2, 3), (3, 5)]

A problem that needed to be navigated was that if a cell became trapped (surrounded by black cells), the node would not come up in the points list. Using the code below counteracts this:

```
def max_value(inputlist):
    return max([sublist[-1] for sublist in inputlist])

goal = max_value(filtered_nodes)

list1 =[i for i in range(goal +1)]
for point in list1:
    if (point in itertools.chain(*filtered_nodes)):
        pass
    else:
        new_node = (point,point)
        filtered_nodes.append(new_node)
```

To make this simulation scalable we must make the goal and MATRIX_SIZE adjustable. For now our final point in our maze is always the bottom corner. So our goal will always be the highest value in our filtered_node list, and our matrix will always be one more than this.

Now we have initiated our environment and know comprehensively how it works. It is now simple to place our Q-learning and Dijkstra powered agents into this environment. Although we as a user can clearly see the environment we must remember that the agents only learn the environment via actions. The code used for Dijkstra's algorithm in the artefact is adapted from Klochay (Klochay, 2021) and the Q-learning code adapted from week 4's materials on blackboard.

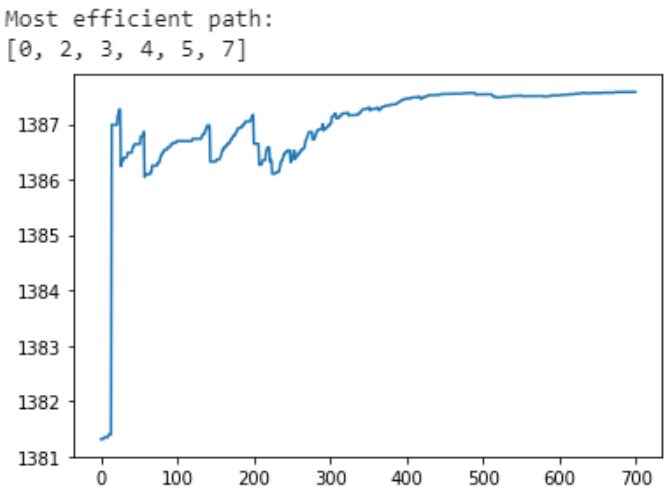
How does Q-learning and Dijkstra apply to our environment

Now that we understand how Q-learning and Djkstra work and we have a workable environment, let's apply our scenario and compare the two different approaches.

For the first scenario, allowing both algorithms to run to their extent to calculate the optimum route; Dijkstra is not only quicker but more space efficient. For a naive implementation of Dijkstra the time complexity is O(V^2) (where V is the number of vertices). Further to this, in the study “An improved Dijkstra’s shortest path algorithm for sparse network “ (Huang,Liu,Luan,Zhang,2017) using fibonacci heaping, this efficiency can be improved (The case study plotted up to 21,000 nodes).

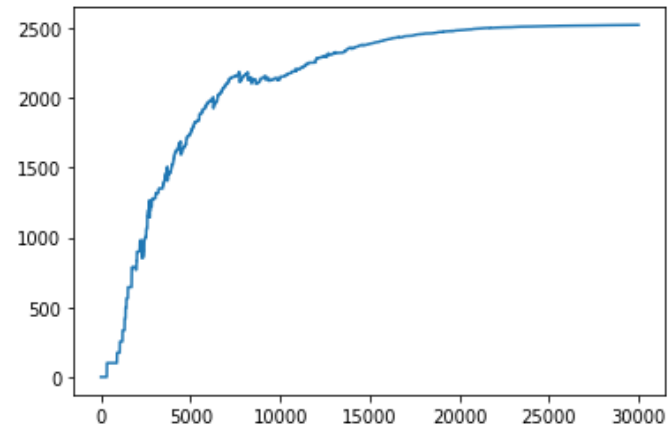
On page 9 are some maze examples of varying sizes with Q-learning implemented, the corresponding graphs show how many “episodes” to return the optimum route. For a 20 by 20 grid it took almost 20000 iterations to return the optimal, which if you imagine a robot moving in real time would take a very long time.

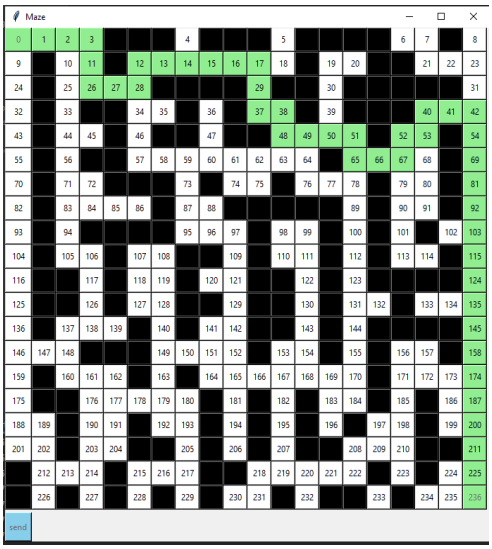
But these figures are not completely accurate for our real-life scenario. Our code at the moment only marks “non-viable” routes as a negative reward but it is still possible to move to that cell and continue to search for the exit. For example in the first figure on page 9 the agent can jump from 0 to 102 then continue to the exit. Although this works perfectly fine to return the optimum route (with enough iterations) it does not work in real life. Instead there are a couple of different methods we could use to code out this “mistake”. One way is to alter the reward scheme; when creating the matrix from the “filtered_nodes” we increase the negative reward to -100 (this will need to be more if there are more that 100 cells since each movement costs -1) and include an exception that if the agent exceeds -100 then the episode is ended prematurely and the agent must return to (0,0) and start the next episode.



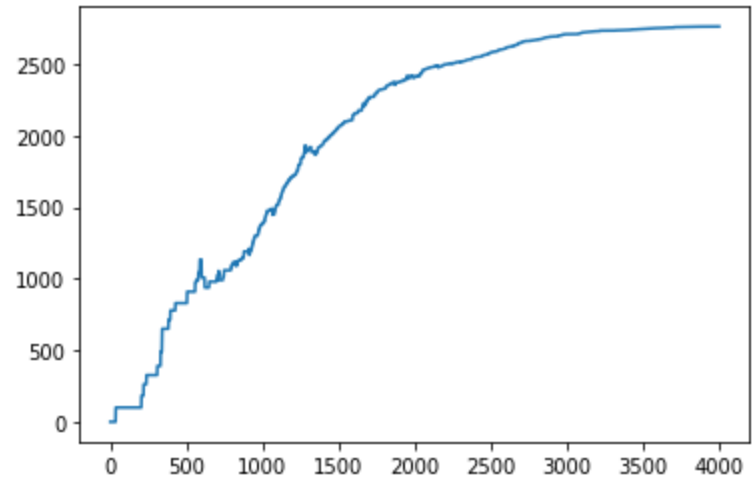
For the second scenario, where the first available exit route is returned as soon as it’s found, both dijkstra and q-learning are fairly similar, and both could accidentally stumble upon the exit. Because of the stochastic nature of the q-learning algorithm, particularly with very large mazes, there is a chance to find an exit fairly quickly. Here you can see an example of Q -learning stumbling across the exit early.

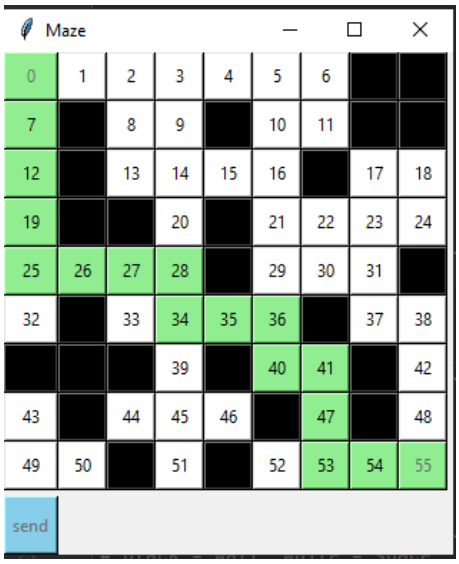
Most efficient path:
[0, 1, 2, 10, 11, 26, 27, 28, 34, 46, 57, 58, 59, 60, 61,



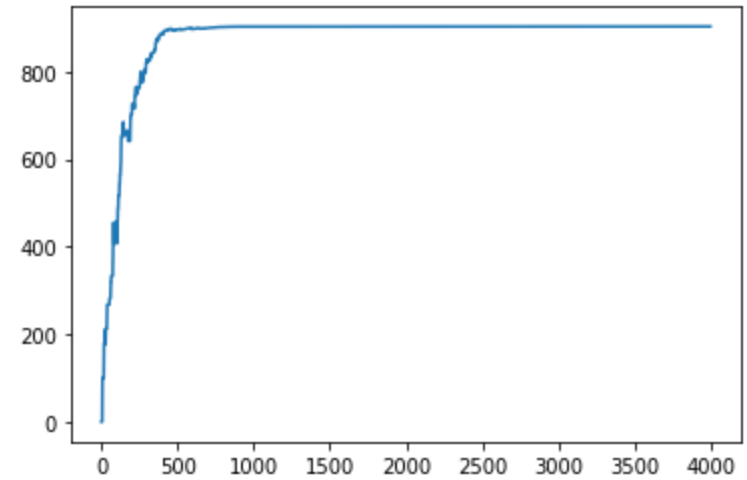


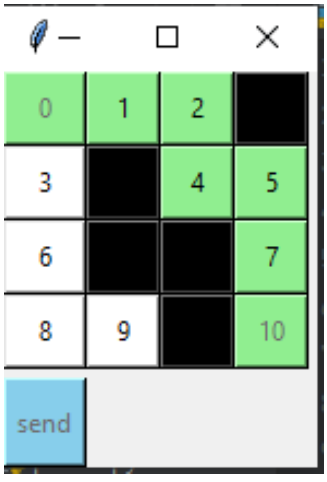
Most efficient path:
[0, 7, 12, 19, 25, 26, 27, 28, 34, 35, 36, 40, 41, 4





Most efficient path:
[0, 1, 2, 4, 5, 7, 10]



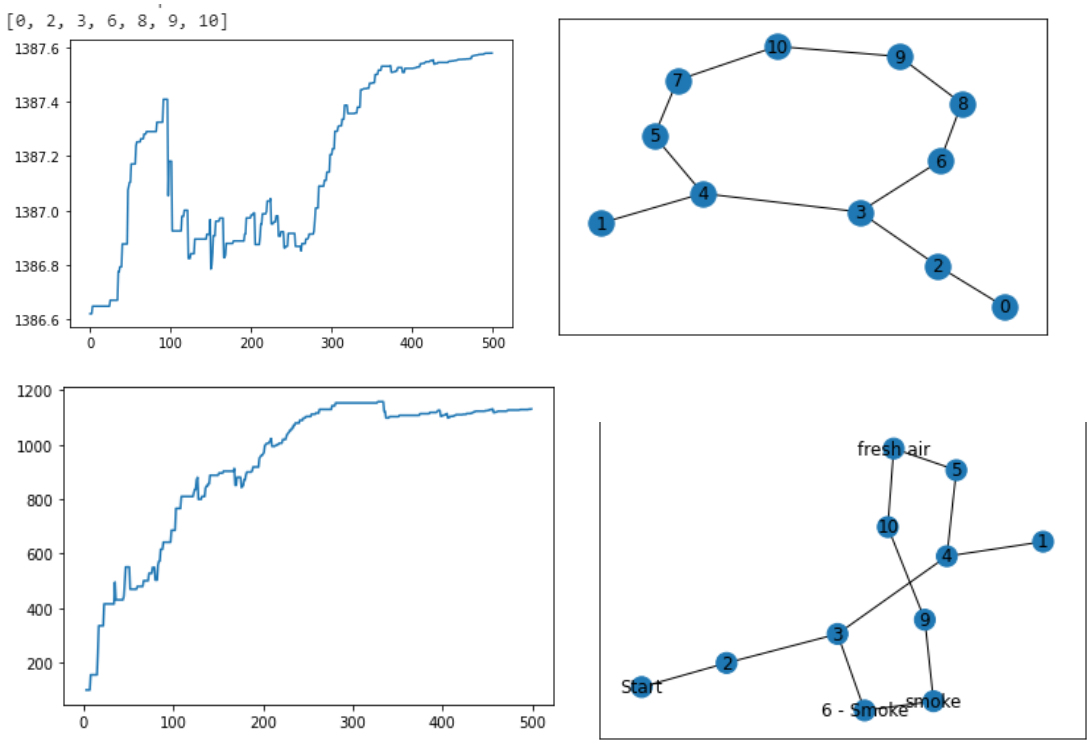


How can we adapt these algorithms to be more efficient or suitable to our specific environment?

One method that can be applied to both algorithms is the way our graph is formed, as it stands, each cell is a node on our graph. We can reduce the number of nodes by only placing a node when the agent reaches a corner (a key decision point) where the agent might have to change direction - this is called a waypoint. This means that if there is a large open area what would originally be hundreds of nodes can be reduced to a few.

We can refine Dijkstra's algorithm further when finding the optimum path, so far we have not utilised its ability to take edge weighting into account. This could be incredibly beneficial as, for example, if the building was rough terrain and would take longer to traverse, then the algorithm compounds time travelled for each node visited, therefore finding the quickest route, not necessarily the shortest.

Another approach that would make Q-learning more advantageous as the method of choice, is the ability to create insight by environmental markers . The program itself does not not know what is inherently “good” or “bad” but we program this. For example if the robot were equipped with sensors that could detect the surrounding air quality. If certain nodes in the graph (cells in the environment) are filled with smoke then we apply a more negative reward to the model, deterring the agent to continue down that path. We can also apply the opposite, the cleaner the surrounding air the better the reward. Below is a small example of the algorithm running first without knowing that smoke is “bad” and air is “good”, the optimum path isn’t found until almost the 400th iteration compared to the model underneath it finding the optimum route at about the 300th iteration.



Changeable environment

Now we have assessed how both algorithms work in a constant environment. Can they be adaptable for a changing environment? For example, if a chunk of debris falls from the ceiling (without destroying the robot) is there a way to manipulate the program for a dynamic environment? For example after visiting a cell the reward function is changed. M. Pieters and M. A. Wiering explore this in “Q-learning with experience replay in a dynamic environment” (M. Pieters and M. A. Wiering, 2016) , they state that Q-learning is not effective in a dynamic environment, but by developing Q-learning by “using a smart exploration policy” and “with experience replay”(M. Pieters and M. A. Wiering, 2016) methodology, it can be achieved efficiently.

Could we apply both algorithms in a three dimensional space?

Dijkstra's algorithm can quite logically be applied in a 3 dimensional environment, a specific example can be found in the paper “A Three-Dimensional Dijkstra’s algorithm for multi-objective ship voyage optimization” (Wang, Eriksson, Mao, 2019). Logically the same can be done for Q-learning.

Conclusion

In practicality, if we are trying to find an escape route for an environment, ideally we would not want to send expensive robots into a dangerous environment that would have to run thousands of loops round the environment. One method that could work well would be to send a robot in to scan the room to produce a greyscale image of the room including obstacles and the robot could then apply Dijkstra's algorithm. There are many other projects that solely focus on robots navigating an unknown environment like the example seen in ‘Distributed Search and Rescue with Robot and Sensor Teams’ (Kantor et al., 2006) . But we must assume this is not possible for what has been discussed in this paper. To summarise, at face value, Dijkstra is more time and space efficient than Q-learning and also has the ability to use weighted edges to emulate longer traversal time. On the other hand, when we consider environmental factors such as smoke levels in the room Q-learning has an advantage. Using negative and positive rewards can speed up the time it would take for the Q-learning algorithm to find the exit and end its first episode. Overall making Q-learning the most suitable algorithm for a changeable environment

Bibliography

Bird, B. *et al* (2019) "Using Autonomous Radiation-Monitoring Assistance to Reduce Risk and Cost", IEEE Robotics & Automation Magazine, volume 26, no. 1, pp. 35-43, doi: 10.1109/MRA.2018.2879755.

Di Stefano, Alessandro (2021) “Week 4 – Reinforcement Learning in Python”. University of Teesside Blackboard.

Frana, P.L. and Misa, T.J. (2010) ‘An interview with Edsger W. Dijkstra’, *Communications of the ACM*, 53(8), pp. 41–47. doi:10.1145/1787234.1787249.

Huang Q.L, Liu Y.Q, Luan G.F, Xu M.H, Zhang Y.X.(2007)”An improved Dijkstra’s shortest path algorithm for sparse network,”. Volume 185, Issue 1, Pages 247-254. Accessed at (https://doi.org/10.1016/j.amc.2006.06.094.)

Kantor, G. *et al.* (2006) ‘Distributed Search and Rescue with Robot and Sensor Teams’, in Yuta, S. et al. (eds) *Field and Service Robotics: Recent Advances in Reserch and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 529–538. doi:10.1007/10991459_51.

Klochay, A. (2021). ”Implementing Dijkstra’s Algorithm in Python. Udemy”. Available at https://www.udacity.com/blog/2021/10/implementing-dijkstras-algorithm-in-python.html (Accessed 01/01/22)

Pieters,M and Wiering, M. A.(2016) "Q-learning with experience replay in a dynamic environment," IEEE Symposium Series on Computational Intelligence (SSCI),, pp. 1-8, doi: 10.1109/SSCI.2016.7849368.

Wang H ,Mao W, Eriksson L.(2019). “*A Three-Dimensional Dijkstra’s algorithm for multi-objective ship voyage optimization” Elsevier Enhanced Reader* doi:10.1016/j.oceaneng.2019.106131.

Watkins, Christopher.(1989).”Learning From Delayed Rewards”. Kings College, Cambridge University UK